Workshop 8: Machine Learning in R

tidymodels

tidymodels is a collection of R packages for **modelling** and **machine learning** that share common step-by-step processes.

Modelling: Supervised ML

Supervised ML models are trained on **labelled data**, learning to map inputs to known outputs.

Create Training and Testing Datasets: rsample

We can train our model on one dataset and test on another dataset that the model hasn't seen before: this gives us an unbiased estimate of real-world performance.

```
df_split <- initial_split(df, prop =
0.75, strata = group)
df_train <- training(df_split)
df_test <- testing(df_split)
```

Define a pre-processing recipe: recipes

Specify which variables we want to include in our model and if any pre-processing needs to be applied. All expressed as statistical formula. We use ~ . to include all columns in our data.

```
our_recipe <-
recipe(df ~ ., data = df_train) %>%
 # add a pre-processing step to normalise
 step_normalise(all_numeric_predictors())
```

Specify your model, engine and mode: parsnip

E.g. we are using a random forest (model) from the ranger R package (engine) and we want a classification algorithm (mode).

```
rf_model <- rand_forest(trees = 500) %>%
   set_engine("ranger") %>%
   set_mode("classification")
```

Create your workflow: workflows

Add your recipe and model

```
rf_wkfl <- workflow() %>%
    add_recipe(our_recipe) %>%
    add_model(rf_model)
```

Fit your workflow and get predictions: parsnip

Fit the workflow once on the training data and generates predictions on the test data in one step. This is why we need to use the `df_split` dataset.

```
rf_fit <- last_fit(rf_wkfl, df_split)
```

Evaluate your model: tune; yardstick

Collect test set performance metrics to get accuracy (proportion of correctly classified) and AUROC (area under the Receiver Operator Characteristic (ROC) curve)

```
collect_metrics(rf_fit)
collect_predictions(rf_fit)
```

Visualising model performance

Precision-Recall (PR) curves show the trade-off between precision (sensitivity) and recall (specificity): good to visualise how **balanced** the dataset is.

```
rf_fit %>%
    collect_predictions() %>%
    pr_curve(
      truth = group,
      # list each group if more than 2
      .pred_mouse1,
      .pred_mouse2,
      .pred_mouse3
    ) %>%
    autoplot()
```

ROC curves show the true positive rate (sensitivity) vs. the false positive rate (1-specificity). Use the same code as above but instead of `pr_curve()`, use `roc_curve()`.

The **confusion matrix** is a table of the "truth" (rows) and "predictions" (columns).

```
rf_fit %>% collect_predictions() %>%
conf_mat(truth = group, estimate =
.pred_class) %>% autoplot()
```

The **Variable Importance Plot** tells us which variable was the most important to inform our model predictions.

```
rf_fit %>% extract_fit_parsnip() %>% vip()
```